Mini KAAL AI Guidance App

Complete Technical Architecture & End-to-End Logic Guide

Candidate / Developer: Anuj Behra	**Stack:** HTML5, CSS3, Vanilla JS, Node.js, Express, PostgreSQL
Assignment: KAAL AI Full Stack Assessment	**Architecture:** 4-Tier Native Client-Server (No Frameworks)

1. High-Level Architecture Overview

The application is intentionally designed using a clean, 4-tier native architecture with zero frontend build dependencies. The Express.js backend acts both as a REST API provider and a static file server for the pure HTML/CSS/JS presentation layer.

Tier	Component	Role & Technology
Tier 1: Frontend	index.html style.css	Semantic HTML5 input form, responsive card layout, error alerts, latest guidance card, and past questions list.
Tier 2: API Bridge	script.js	Vanilla JavaScript event listeners, input trimming, and browser-native asynchronous fetch() requests.
Tier 3: Backend	server.js	Node.js & Express server handling JSON parsing, input validation, guidance logic, and REST routing.
Tier 4: Persistence	db.js (PostgreSQL)	Connection pool (pg.Pool) executing parameterized SQL queries; auto-creates queries table.

2. End-to-End Step-by-Step Flow (Lifecycle of a Question)

When a user enters a question and clicks Submit, data travels through a disciplined 10-step sequence across the client, server, and database:

Step 1: User Input: The user types a question into <textarea id='question-input'> and clicks <button type='submit'>.

Step 2: Event Interception: script.js catches the submit event and executes e.preventDefault() to prevent a full page reload.

Step 3: Client Validation: Leading and trailing whitespace is stripped via .trim(). If empty, an error message is displayed immediately without sending an HTTP request, saving bandwidth.

Step 4: UI State Management: The submit button is disabled and its label changes to 'Submitting...'; the loading box is unhidden. This prevents double-click submission races.

Step 5: Network Request (Fetch): The browser initiates a POST request to /api/ask carrying a serialized JSON payload: JSON.stringify({ question }) with Content-Type: application/json.

Step 6: Server Body Parsing & Validation: Express parses the request body via express.json(). The server checks that question is non-empty. Never trusting the client is a fundamental security practice.

Step 7: Guidance Generation (AI + Dummy Fallback): The server calls getGuidanceResponse(question). If GEMINI_API_KEY is set, it queries Google Gemini. If omitted or upon network failure, it seamlessly matches keywords (career, study, stress, code) to return structured, encouraging advice.

Step 8: SQL Insertion & Parameterization: The backend executes INSERT INTO queries (question, response) VALUES ($1, $2) RETURNING *. Placeholders ($1, $2) guarantee immunity against SQL Injection attacks.

Step 9: Server Response (HTTP 201): PostgreSQL generates an auto-incremented id and exact created_at timestamp. Express returns the saved record as JSON with status 201 Created.

Step 10: DOM Update & History Refresh: The frontend updates the response card with the question, answer, and formatted date. It clears the textarea and calls fetch('/api/history') to render the newest query at the top of the history list.

3. Detailed Component & Code Breakdown

Database Layer (backend/db.js)

Initializes PostgreSQL via the official pg connection pool. Runs an automated table creation check on startup so evaluators do not need to execute manual SQL migration scripts.

```
const { Pool } = require('pg'); require('dotenv').config(); const isCloudDB =
process.env.DATABASE_URL && !process.env.DATABASE_URL.includes('localhost'); const pool =
new Pool({ connectionString: process.env.DATABASE_URL ||
'postgresql://localhost:5432/postgres', ssl: isCloudDB ? { rejectUnauthorized: false } :
false }); const initDB = async () => { try { await pool.query(` CREATE TABLE IF NOT EXISTS
queries ( id SERIAL PRIMARY KEY, question TEXT NOT NULL, response TEXT NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ); `); console.log('PostgreSQL connected
and "queries" table is ready!'); } catch (err) { console.error('Database connection
error:', err.message); } }; module.exports = { pool, initDB };
```

Server Layer (backend/server.js)

Serves static frontend files, exposes REST endpoints, and manages the dual-mode guidance engine.

```
// REST Endpoint 1: Fetch History app.get('/api/history', async (req, res) => { try {
const result = await pool.query('SELECT * FROM queries ORDER BY created_at DESC');
res.json(result.rows); } catch (err) { res.status(500).json({ error: 'Failed to fetch
history from database' }); } }); // REST Endpoint 2: Submit Question app.post('/api/ask',
async (req, res) => { try { const { question } = req.body; if (!question ||
question.trim() === '') { return res.status(400).json({ error: 'Please enter a question
before submitting.' }); } const cleanQuestion = question.trim(); const response = await
getGuidanceResponse(cleanQuestion); const query = 'INSERT INTO queries (question,
response) VALUES ($1, $2) RETURNING *'; const result = await pool.query(query,
[cleanQuestion, response]); res.status(201).json(result.rows[0]); } catch (err) {
res.status(500).json({ error: 'Something went wrong on the server' }); } });
```

4. Technical Terminology & Core Concepts

Term	Plain-English Definition & Context in This Project
Client-Side vs Server-Side	**Client-Side:** Code running in the browser (HTML, CSS, JS) responsible for display and user interaction. **Server-Side:** Code running on Node.js responsible for business logic, security, and database communication.
REST API	Representational State Transfer. A standard pattern using HTTP verbs (GET to read, POST to create) and standard JSON payloads to exchange data between frontend and backend.
Connection Pool (pg.Pool)	Instead of opening and closing an expensive new database connection for every incoming user request, a connection pool keeps a cache of reusable connections open, dramatically speeding up throughput.
Parameterized Queries	Using variables ($1, $2) instead of string concatenation. PostgreSQL treats input values strictly as data, never as executable code, preventing SQL Injection vulnerabilities.
Event Loop & Async/Await	JavaScript is single-threaded. async/await allows Node.js to wait for database disk I/O and network API calls in the background without blocking other incoming user requests.
CORS	Cross-Origin Resource Sharing. A browser security rule that restricts web pages from requesting a domain different from the one that served it. The cors() middleware enables controlled API access.
XSS Protection	Cross-Site Scripting. An attack where users inject malicious JavaScript into web forms. The app mitigates this by injecting strings into text nodes and encoding HTML entities.

5. Architectural Justifications & Problem-Solving (Interview Prep)

Q1: Why did you choose Vanilla JavaScript instead of React?
Answer: For a focused guidance application with two cards and a history list, React introduces heavy build tooling (Vite/Webpack), hundreds of megabytes of node_modules, and separate development ports. Vanilla JavaScript allows the Express server to directly serve the frontend via express.static(), booting the entire full-stack app with a single npm start command with zero compilation overhead.

Q2: Why use raw PostgreSQL with pg.Pool instead of an ORM like Prisma or Sequelize?
Answer: For a single table domain model, an ORM adds unnecessary abstraction, runtime overhead, and complex migration engines. Direct SQL with connection pooling provides complete transparency, shows raw database proficiency, and eliminates SQL injection risks via parameterization.

Q3: What was a practical technical challenge you encountered and resolved?
Answer: On macOS Monterey and later, Apple's AirPlay Receiver (ControlCenter) listens on port 5000 by default, triggering an EADDRINUSE port collision. I configured the server to run on port 5001 by default and designed dynamic client URL resolution so the app runs smoothly locally and in cloud production.

Q4: How does the application guarantee 100% uptime if no AI API key is provided?
Answer: The guidance generator implements a graceful fallback pattern. It attempts to query Google Gemini if an API key is configured in .env. If omitted or if an error occurs, it smoothly routes to contextual keyword-based guidance rules, ensuring evaluators can test the project without API key blockers.